

Санкт–Петербургский государственный университет

ТРЕТЬЯКОВ Даниил Александрович

Выпускная квалификационная работа

Исследование эффективности непрерывного обучения с использованием сетей Колмогорова-Арнольда в задаче распознавания образов

Уровень образования: бакалавриат

Направление 01.03.02 «Прикладная математика и информатика»

Основная образовательная программа СВ.5005.2021 «Прикладная математика, фундаментальная информатика и программирование»

Научный руководитель:

профессор кафедры компьютерного моделирования и многопроцессорных систем, д.т.н. Дегтярев Александр Борисович

Рецензент:

доцент кафедры компьютерного моделирования и многопроцессорных систем, к.ф.-м.н. Корхов Владимир Владиславович

Санкт-Петербург

2025 г.

Содержание

Введение	4
Постановка задачи	7
Обзор литературы	10
Глава 1. Анализ существующих подходов к преодолению катастрофического забывания	12
1.1. Подход с использованием регуляризации	13
1.1.1 Весовая регуляризация (Prior Focused)	13
1.1.2 Функциональная регуляризация (Data Focused)	15
1.2. Подход с изменением архитектуры	16
1.2.1 Фиксированная архитектуры (Fixed Network)	16
1.2.2 Динамическая архитектура (Dynamic Network)	18
1.3. Подход с воспроизведением опыта	19
1.3.1 Повторение (Rehearsal)	19
1.3.2 Псевдоповторение (Pseudo Rehearsal)	20
1.3.3 С градиентными ограничениями (Constrained)	21
Глава 2. Построение усовершенствованных моделей на базе архитектуры KAN	23
2.1. Подход с использованием регуляризации	26
2.2. Подход с изменением архитектуры	28
2.3. Подход с воспроизведением опыта	28
Глава 3. Применение усовершенствованных моделей в задачах непрерывного обучения	31
3.1. Обучение с перестановкой признаков	31
3.2. Инкрементальное обучение классам	31
Глава 4. Анализ экспериментов	32
Выводы	33
Заключение	34
Список литературы	35
Приложения	39

Введение

В последние десятилетия нейронные сети стали одним из ключевых компонентов современных интеллектуальных систем. Именно нейронные сети сделали возможным появление алгоритмов, превосходящих человеческие возможности в узкоспециализированных задачах, где люди традиционно считались непревзойденными, например, в распознавании изображений и речи, играх и стратегиях, медицинской диагностике.

Однако в отличие от человека, для нейронной сети, после того как она была обучена выполнению определенной задачи, например, классификации деревьев, обучение дополнительной задаче, например, классификации птиц, оказывается затруднительным. Это связано с тем, что при добавлении новой задачи типичная нейронная сеть склонна «забывать» предыдущую. В связи с этим, одним из актуальных направлений в развитии нейронных сетей является непрерывное обучение - подход, при котором модель способна обучаться и адаптироваться на протяжении всего срока эксплуатации с целью поддержания своей производительности при изменяющихся условиях окружающей среды и данных, без необходимости полного переобучения. Потребность в подобных моделях особенно сильна в рекламной индустрии, финансовой аналитике, робототехнике и кибербезопасности.

Как правило, в исследованиях, посвященных методам преодоления катастрофического забывания, рассматривается многослойный перцептрон (Multi-Layer Perceptron, MLP). Несмотря на то, что в борьбе с катастрофическим забыванием в многослойном перцептроне были достигнуты определенные успехи, в последние месяцы растёт интерес к многообещающей альтернативе - нейронной сети Колмогорова-Арнольда (Kolmogorov-Arnold Network, KAN), ключевые отличия которой представлены на рисунке 1.

Главная причина, по которой KAN представляет интерес с точки зрения применения в условиях непрерывного обучения, заключается в возможности параметризации функции активации таким образом, чтобы изменение параметров в одной области пространства не влияло на другие области. Это возможно, например, при использовании B-сплайнов, обладающих локальной поддержкой. Поскольку обучение на новых данных будет затрагивать лишь

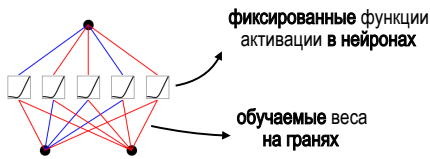
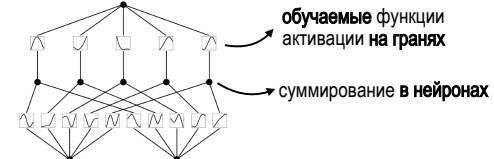
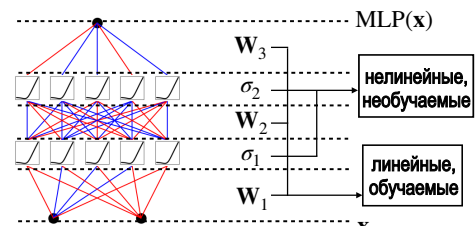
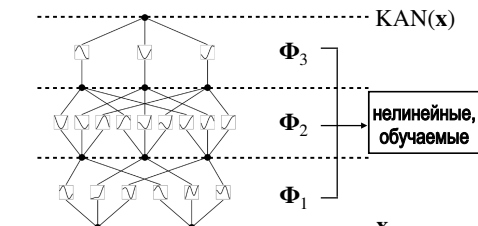
Модель	Multi-Layer Perceptron (MLP)	Kolmogorov-Arnold Network (KAN)
Теорема	Универсальная теорема аппроксимации	Теорема о представлении Колмогорова-Арнольда
Формула слоя	$f(\mathbf{x}) \approx \sum_{i=1}^{M(\epsilon)} a_i \sigma(\mathbf{w}_i \cdot \mathbf{x} + b_i)$	$f(\mathbf{x}) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$
Структура слоя		
Формула сети	$\text{MLP}(\mathbf{x}) = (\mathbf{W}_3 \circ \sigma_2 \circ \mathbf{W}_2 \circ \sigma_1 \circ \mathbf{W}_1)(\mathbf{x})$	$\text{KAN}(\mathbf{x}) = (\Phi_3 \circ \Phi_2 \circ \Phi_1)(\mathbf{x})$
Структура сети		

Рис. 1: Сравнение MLP и KAN. Адаптировано из [11], лицензия: CC BY 4.0.

несколько соседних параметров, информация, сохранённая в остальных параметрах, остается нетронутой. В MLP, напротив, даже отдельные изменения могут повлиять на информацию, хранящуюся в отдалённых друг от друга параметрах.

Для демонстрации преимуществ, связанных с локальной поддержкой функции активации, предлагается рассмотреть простейшую задачу одномерной регрессии, детали которой приведены на рисунке 2: обучение состоит из 5 тренировочных сессий, в каждой из которых используются исключительно объекты из соответствующей части набора данных, также разделённого на 5 частей. Можно заметить, что в данной задаче KAN изменяет только те параметры, которые отвечают за аппроксимацию зависимости в данных текущей тренировочной сессии, в то время как MLP оказывается не способен сохранить информацию, полученную в течение предыдущих тренировочных сессий. Рассмотренная задача является лишь упрощённым примером, в котором особенности архитектуры KAN позволили существенно повлиять на эффективность непрерывного обучения. В прикладных задачах размерность параметрического пространства нейронных сетей, как правило, значительно выше, и сохранение свойства локальности изменений параметров оказывает-

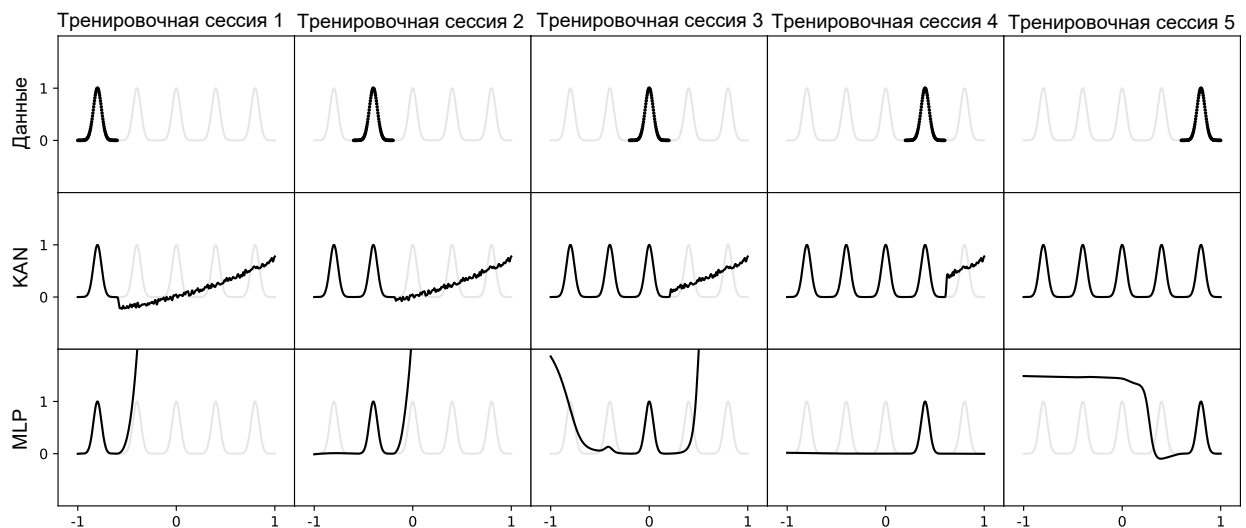


Рис. 2: Простейшая задача непрерывного обучения: по горизонтали показаны 5 последовательных тренировочных сессий одномерной регрессии («Тренировочная сессия 1-5»); по вертикали — три строки: «Данные» (данные, используемые в текущей тренировочной сессии, выделены чёрным, остальные серым), «KAN» (предсказания сети KAN) и «MLP» (предсказания сети MLP). Адаптировано из [11], лицензия: CC BY 4.0.

ся под вопросом.

Поскольку необходимым условием для достижения максимальной эффективности непрерывного обучения является использование нейронных сетей в сочетании с методами противодействия катастрофическому забыванию, тестирование оригинальной архитектуры KAN без этих методов не позволит в полной мере оценить перспективы её применения в непрерывном обучении. Поэтому главная цель данной работы - адаптировать методы противодействия, разработанные для многослойного перцептрона, для использования в сетях Колмогорова-Арнольда и выяснить, позволяет ли локальная природа KAN упростить или усилить классические приёмы, лежащие в основе этих методов. С этой целью эффективность полученных моделей исследуется в условиях непрерывного обучения на задаче распознавания образов с использованием набора данных CUB200, обладающего свойствами, характерными для прикладных задач.

Полный исходный код, а также графические материалы, использованные в данной работе, размещены в открытом репозитории на GitHub [27].

Постановка задачи

Рассматривается процесс непрерывного обучения в задаче многоклассовой классификации в признаковом пространстве $\mathcal{X} \subseteq \mathbb{R}^d$ с множеством классов $\mathcal{Y} = \{1, \dots, C\}$, где d - размерность признакового пространства, C - число классов. Дана обучающая выборка D , состоящая из размеченных объектов, разбитых на T частей, соответствующих тренировочным сессиям B_t , то есть $D = \{B_t\}_{t=1}^T$. Каждая тренировочная сессия состоит из N_t объектов (N_t может изменяться между сессиями). Таким образом, $B_t = \{(x_i, y_i)\}_{i=1}^{N_t}$, где $x_i \in \mathcal{X}$, $y_i \in \mathcal{Y}$. Предполагается, что элементы в каждой тренировочной сессии независимы и имеют одинаковое совместное распределение данных и меток классов $\mathbb{D}_t$, $t = 1, \dots, T$. Нейронная сеть $f_\theta : \mathcal{X} \rightarrow \mathbb{R}^C$ с параметрами $\theta \in \mathbb{R}^{|\theta|}$ последовательно обучается решению T задач, используя объекты текущей тренировочной сессии. Иначе говоря, в момент времени t модели предоставлены лишь данные из B_t .

Требуется получить модель f_θ , способную сохранять информацию, полученную в предыдущих тренировочных сессиях обучения, и одновременно эффективно усваивать новые знания, приобретённые в течение последней тренировочной сессии. Для оценки качества обучения модели в текущей тренировочной сессии используется следующий функционал:

$$Q(f_\theta, B_t) = \frac{1}{N_t} \sum_{(x,y) \in B_t} \mathcal{L}(f_\theta(x), y),$$

где $\mathcal{L}$ - кросс-энтропийная функция потерь. Параметры модели θ подбираются таким образом, чтобы минимизировать функционал Q . Обучение сводится к нахождению оптимальных параметров θ_t , которые сохраняются после завершения текущей тренировочной сессии и используются в качестве начальных значений в следующей:

$$\theta_t = \arg \min_{\theta} Q(f_\theta, B_t) = \arg \min_{\theta} \frac{1}{N_t} \sum_{(x,y) \in B_t} \mathcal{L}(f_\theta(x), y),$$

Для количественной оценки способности моделей противостоять катастрофическому забыванию в процессе непрерывного обучения используются

метрики, описанные в [20]:

$$\Omega_{base} = \frac{1}{T-1} \sum_{i=2}^T \frac{\alpha_{base,i}}{\alpha_{ideal}},$$

$$\Omega_{new} = \frac{1}{T-1} \sum_{i=2}^T \frac{\alpha_{new,i}}{\alpha_{ideal}},$$

$$\Omega_{all} = \frac{1}{T-1} \sum_{i=2}^T \frac{\alpha_{all,i}}{\alpha_{ideal}},$$

где:

α_{ideal} - значение метрики ассигасу на совокупности всех тестовых выборок D , полученное при оффлайн обучении модели без механизмов противодействия катастрофическому забыванию,

$\alpha_{base,i}$ - значение метрики ассигасу на тестовой выборке первой тренировочной сессии после обучения модели в i -ой тренировочной сессии,

$\alpha_{new,i}$ - значение метрики ассигасу на тестовой выборке i -ой тренировочной сессии после обучения модели на этой же тренировочной сессии,

$\alpha_{all,i}$ - значение метрики ассигасу на совокупности тестовых выборок всех предыдущих тренировочных сессий.

Таким образом, Ω_{new} измеряет способность модели использовать только что приобретенные знания, Ω_{base} измеряет способность модели удерживать в памяти знания, полученные в первой тренировочной сессии, а Ω_{all} оценивает общую эффективность использования накопленных знаний.

Цель и задачи исследования

Целью данной работы является исследование эффективности применения архитектуры KAN в сочетании с методами противодействия катастрофическому забыванию в задаче распознавания образов в условиях непрерывного обучения. Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить методы противодействия катастрофическому забыванию, разработанные для многослойного перцептрона, и выделить основные под-

ходы, лежащие в основе этих методов.

2. Адаптировать для архитектуры KAN методы, использующие разные подходы, и разработать на их основе модифицированные модели.
3. Применить оригинальную и модифицированные модели KAN в задаче распознавания образов в условиях непрерывного обучения с использованием набора данных CUB200.
4. Сравнить противодействие катастрофическому забыванию между оригинальной и модифицированными моделями и выяснить, помогает ли свойство локальной поддержки функций активации KAN повысить эффективность классических приёмов, использованных в этих моделях.
5. Проанализировать полученные результаты и оценить перспективы применения сетей Колмогорова-Арнольда в задачах непрерывного обучения.

Обзор литературы

Нейронная сеть Колмогорова-Арнольда, составляющая основу данного исследования, описана в статьях [10; 11], где вводятся архитектуры KAN и MultKAN, позиционируемые в качестве альтернативы MLP. В [11] поднимается вопрос о необходимости исследования потенциала сетей Колмогорова-Арнольда в непрерывном обучении на задачах высокой размерности, также в обеих статьях проводятся многочисленные эксперименты, демонстрирующие интерпретируемость новых архитектур и их применимость в качестве инструмента символьной регрессии. Всестороннее сравнение архитектур KAN и MLP представлено в работе [28]. В ней делается вывод о том, что KAN демонстрирует преимущество только в задаче символьной регрессии, а в остальных задачах машинного обучения, включая задачи компьютерного зрения (Computer Vision, CV), обработки естественного языка (Natural Language Processing, NLP) и обработки аудиосигналов (audio processing), уступает. В частности, показано, что в условиях непрерывного обучения официальная реализация KAN [18] полностью теряет первоначальные знания после обучения всего на трёх последовательных задачах, тогда как MLP в тех же условиях сохраняет приемлемую точность. Выводы этой статьи обусловили смещение акцентов в настоящем исследовании в сторону анализа KAN в сочетании с классическими методами противодействия катастрофическому забыванию, а не самой по себе оригинальной архитектуры, и кроме этого подтолкнули к внесению изменений в официальную программную реализацию.

Среди исследований, посвящённых непосредственно явлению катастрофического забывания в сетях Колмогорова-Арнольда следует выделить статью [12]. Её основным результатом является архитектура WiseKAN, но наибольший интерес представляют сформулированные в ней причины обострения проблемы катастрофического забывания в KAN. Первая причина - пересечение входных значений сплайна, вторая - низкая разреженность градиентов вследствие необходимости поиска компромисса между шириной слоёв нейронной сети и количеством параметров в сплайнах. Эти выводы показали необходимость рассмотрения в данном исследовании методов, основанных на использовании разреженного кодирования представлений.

Теме непрерывного обучения посвящено множество научных публикаций, однако количество монографий невелико. Изучение теоретического материала базировалось на книге [3], в которой представлен систематический обзор парадигм машинного обучения (в частности, непрерывного обучения), а также книге [19], где рассматриваются проблемы, возникающие при обработке нестационарных данных. Анализ современного состояния методов противодействия катастрофическому забыванию опирался на ряд специализированных обзоров: в [20] предложена таксономия, разделяющая подходы на регуляризацию, ансамблирование, разреженное кодирование и воспроизведение опыта. В [5] проведена параллель между искусственными нейронными сетями и мозгом, а также сгруппированы и рассмотрены основные биологические механизмы, приведшие к появлению новых алгоритмов непрерывного обучения. В [1] даётся цельный обзор: формулируются постановка задачи и теоретические основы разных подходов, а также предлагается расширенная таксономия методов и анализируется их применение в широком спектре областей (CV, NLP и другие), что делает работу актуальным срезом состояния области на 2024 год.

В результате изучения научных публикаций по теме непрерывного обучения был сформирован окончательный список методов, подлежащих адаптации для нейронных сетей Колмогорова-Арнольда: Fixed Expansion Layer (FEL), GeppNet и Elastic Weight Consolidation (EWC). Основные сведения, необходимые для их реализации, были получены из соответствующих статей [6; 8; 22], однако обновление информационной матрицы Фишера в EWC было изменено в соответствии с выводами [24], а самоорганизующаяся карта (Self-organizing map, SOM) в GeppNet заменена на Batch-SOM ([4]). Подробности реализации этих методов описаны в главе 2.

Глава 1. Анализ существующих подходов к преодолению катастрофического забывания

Поскольку явление, приводящее к потере моделью ранее выученной информации, получившее название катастрофического забывания, является основным вызовом в области непрерывного обучения, для борьбы с ним разработаны многочисленные методы, которые принято делить на несколько основных подходов. Данная работа опирается на двухуровневую схему, предложенную в [13], где выделяются три основных подхода, а также их подкатегории:

1. **Использование регуляризации:** в функцию потерь добавляются регуляризирующие члены, призванные сохранить значения важных параметров, связанных с предыдущими задачами.
2. **Изменение архитектуры:** структура нейронной сети расширяется или изменяется с целью назначения для каждой из задач отдельного набора параметров.
3. **Воспроизведение опыта:** образцы данных, встречаемых в процессе обучения, сохраняются или используются для обучения генеративной модели. Затем эти образцы (или псевдо-образцы, если использовалась генеративная модель) используются для освежения знаний модели о старых задачах.

Полная схема разбиения методов представлена на рисунке 3.

Чтобы наглядно сравнить то, как свойство локальной поддержки функций активации в KAN взаимодействует с каждой из принципиально отличающихся друг от друга стратегий противодействия катастрофическому забыванию, было принято решение адаптировать по одному методу из каждого подхода. Несмотря на упоминание в литературе современных (как правило, гибридных) методов, предпочтение отдавалось классическим алгоритмам. Они имеют прозрачную теоретическую основу и достаточно просты для переноса в новую архитектуру, однако по-прежнему выступают в качестве базовых моделей в большинстве современных исследований.



Рис. 3: Двухуровневая таксономия методов противодействия катастрофическому забыванию, предложенная в [13]. Во второй строке показаны три базовых подхода: использование регуляризации (regularization-based), изменение архитектуры (architecture-based) и воспроизведение опыта (replay-based). Ниже перечислены подкатегории, составляющие каждый из подходов.

Оставшаяся часть главы посвящена описанию принципов, лежащих в основе выделенных подходов, а также обзору алгоритмов, которые их реализуют.

1.1 Подход с использованием регуляризации

1.1.1 Весовая регуляризация (Prior Focused)

Основной принцип методов с весовой регуляризацией, таких как Elastic Weight Consolidation (EWC) [22] и Synaptic Intelligence (SI) [29], заключается в использовании новой функции потерь $\mathcal{L}_{total}$, полученной добавлением в стандартную функцию потерь $\mathcal{L}$ нового слагаемого, штрафующего за изменение

тех параметров нейронной сети, которые имели наибольшую важность для выполнения предыдущих задач. В терминах ранее введенных обозначений:

$$\mathcal{L}_{\text{total}}(f_{\theta}(x), y, \theta^{(t-1)}) = \mathcal{L}(f_{\theta}(x), y) + \|\theta - \theta^{(t-1)}\|_{\Sigma},$$

где:

$\theta^{(t-1)}$ - априорные значения параметров, отклонение от которых должно быть ограничено,

Σ - матрица оценок важности параметров с точки зрения их влияния на качество выполнения предыдущих задач,

$\|\cdot\|_{\Sigma}$ - взвешенная норма.

Часто используется L2-норма, с которой регуляризующее слагаемое принимает вид $\frac{1}{2}(\theta - \theta^{(t-1)})^T \Sigma (\theta - \theta^{(t-1)})$. Это можно интерпретировать как наделение весов внутренним состоянием, которое определяет уровень их пластичности. Другой полезной интерпретацией является рассмотрение таких методов в качестве последовательного приближенного Байесовского вывода для параметров нейронной сети.

Одним из ключевых аспектов рассматриваемых методов считается оценка важности параметров. Распространённым подходом является использование информационной матрицы Фишера (Fisher Information Matrix, FIM), которая показывает как небольшие изменения параметров повлияют на значение функции потерь. Как правило, для ускорения вычислительных процедур используется диагональная аппроксимация FIM, что означает предположение о независимости признаков. Вторым существенным недостатком матрицы Фишера оказалась вычислительная затратность. Вместе они привели к появлению целого ряда методов [21; 29], аппроксимирующих FIM или заменяющих её прямой подсчет эвристическими методами.

В рамках данного исследования методы весовой регуляризации представляют значительный интерес, так как отличия в способе параметризации KAN приводят к изменению ландшафта функции потерь, а это может оказать влияние на эффективность подхода с использованием регуляризации.

1.1.2 Функциональная регуляризация (Data Focused)

По аналогии с весовой регуляризацией, функциональная регуляризация добавляет новое слагаемое в функцию потерь $\mathcal{L}$. Однако эффективная и вместе с тем корректная оценка весов модели представляет большую сложность из-за комплексной взаимосвязи между поведением нейронной сети и её параметрами. Из-за этого оказывается оправданным уход от пространства параметров к оперированию ограничениями непосредственно в функциональном пространстве нейронной сети:

$$\mathcal{L}_{\text{total}}(f_{\theta}(x), y, \theta^{(t-1)}) = \mathcal{L}(f_{\theta}(x), y) + \langle f_{\theta}, f_{\theta^{(t-1)}} \rangle_{\mathcal{A}},$$

где:

$\mathcal{A}$ - множество элементов из набора входных данных, выступающих в роли якорных точек (anchor points),

$f_{\theta^{(t-1)}}$ - отображение, сопоставляющее каждому элементу из множества $\mathcal{A}$ (якорных точек) соответствующий выход нейронной сети с параметрами $\theta^{(t-1)}$,

$\langle \cdot, \cdot \rangle$ - мера близости на множестве $\mathcal{A}$.

Таким образом, основная цель функциональной регуляризации состоит в минимизации различий между выходами новой f_{θ} и старой $f_{\theta^{(t-1)}}$ нейронных сетей на множестве специальных элементов входного набора данных $\mathcal{A}$, называемых якорными точками.

Существуют различные способы измерить расхождение между функциями f_{θ} и $f_{\theta^{(t-1)}}$. Наибольшую популярность приобрёл использованный в алгоритме Learning without Forgetting (LwF) [16] подход с дистилляцией знаний [9]. Поскольку расхождение между f_{θ} и $f_{\theta^{(t-1)}}$ не обязательно измерять только в выходном пространстве нейронной сети, в некоторых работах, например [26], применяется дистилляция признаков, при которой для передачи знаний от «учителя» к «ученику» используются не только выходы сети, но и формируемые в скрытых слоях промежуточные представления (признаки).

Ещё одним важным вопросом в функциональной регуляризации является выбор якорных точек $\mathcal{A}$. Формирование этого множества путём случайного

выбора элементов из входного набора данных может оказаться достаточным лишь для относительно простых задач. Одно из решений состоит в использовании текущей тренировочной сессии B_t в качестве $\mathcal{A}$, как в алгоритме LwF, однако такой подход применим только в случае, если данные в разных тренировочных сессиях имеют одинаковую модальность (например, являются изображениями). Существуют подходы, стремящиеся выбрать из каждой тренировочной сессии ограниченный набор репрезентативных в некотором смысле объектов, но вопрос оптимального выбора якорных точек во многом остается открытой проблемой.

Необходимость использовать для выбора якорных точек эвристические методы, способных внести неопределенность в результаты экспериментов из-за отличий между архитектурами KAN и MLP, обусловили отказ от рассмотрения функциональной регуляризации при поиске подходящих для адаптации методов.

1.2 Подход с изменением архитектуры

Рассмотренный ранее подход предназначен для обучения модели выполнению нескольких задач посредством эффективного переиспользования одного и того же набора параметров, задействованного во всех тренировочных сессиях. Однако существует другой подход: назначение для каждой из задач обособленного (возможно, непересекающегося) набора параметров с помощью изолирования их друг от друга в рамках одной модели (фиксированная архитектура) или создания новых моделей, приспособленных для решения отдельных задач (динамическая архитектура). Это позволяет избегать перезаписи информации, содержащейся в параметрах нейронной сети, тем самым снижая воздействие катастрофического забывания.

1.2.1 Фиксированная архитектуры (Fixed Network)

Основным принципом моделей с фиксированной архитектурой является изоляция параметров, используемых в разных тренировочных сессиях, для уменьшения их взаимного влияния на предварительно выученные представления. Среди сильных сторон фиксированных сетей выделяют низкий расход

памяти и относительную простоту. Однако они требуют тщательного планирования при выделении весов под каждую из задач, а также, как и сети с динамической архитектурой, они часто нуждаются в информации о текущем типе задачи в условиях инкрементального обучения задачам (task-incremental learning).

Центральным вопросом в сетях с фиксированной архитектурой является способ разделения весов на группы: следует ли фиксировать группы заранее или позволить им формироваться в процессе обучения; выбирать расположение весов в рамках одной группы случайно или целыми блоками.

Одной из наиболее ранних попыток предложить ответ на эти вопросы стала архитектура PathNet [23]. В нём каждый слой заранее разбивается на M модулей (групп нейронов), а в процессе обучения генетическим алгоритмом ищется оптимальный маршрут (path), представляющий собой вектор индексов, указывающий, какие из $N < M$ модулей в каждом слое будут активны в данной задаче. Передача сигнала происходит только между активными модулями соседних слоёв, поэтому веса остальных нейронов не участвуют ни в прямом (forward pass), ни в обратном (backward pass) проходе и не обновляются в ходе обратного распространения ошибки. Считается, что априорное разбиение параметров на группы, как у PathNet, обеспечивает высокую стабильность, но может приводить к нехватке выразительности сети из-за фрагментации параметрического пространства

В качестве отдельного направления внутри фиксированных сетей следует выделить архитектуру Fixed Expansion Layer (FEL) [6]. Её основная идея заключается в добавлении расширенного слоя с жёстко заданными, необучаемыми весами. Он призван разреживать проходящие через него сигналы, благодаря чему обновление градиента в течение очередной тренировочной сессии затрагивают лишь обособленные группы параметров предыдущего слоя, что резко снижает количество нежелательных изменений в параметрах, ответственных за решения разных задач. В то же время расширенный слой остаётся фиксированным, поэтому в отличие от методов с динамической архитектурой, рассматриваемых далее, его использование не приводит к неограниченному росту числа параметров.

Алгоритмы с фиксированной архитектурой представляют большой ин-

интерес, чем с динамической архитектурой, так как они направлены на локализацию изменений параметров в отдельных группах фиксированного размера, а этот эффект может быть усилен благодаря свойству локальности поддержки функций активации KAN.

1.2.2 Динамическая архитектура (Dynamic Network)

Нейронные сети с динамической архитектурой можно условно разделить на две группы: модели, использующие ансамблевые методы, а также модели, использующие расширение архитектуры (network expansion).

К первой группе относятся алгоритмы, стремящиеся избежать катастрофического забывания посредством обучения нескольких моделей «экспертов» и комбинирования их ответов. Например, Learn++ [15] адаптирует классический алгоритм AdaBoost, наращивая ансамбль после каждой тренировочной сессии и делая итоговое предсказание с помощью взвешенного голосования. Недостатком Learn++ является линейный рост требуемой для хранения весов памяти и увеличение времени инференса. Развитие этого подхода было предложено в [17], где вводится весовая схема, способная одновременно как добавлять, так и исключать экспертов в зависимости от их полезности в последних тренировочных сессиях. Особое место среди трудностей, возникающих при разработке методов, представляющих собой ансамбль экспертов, занимает необходимость выбора эксперта для получения предсказания.

В свою очередь, модели, использующие расширение архитектуры, в каждой тренировочной сессии добавляют в изначальную архитектуру новые нейроны или целые слои, а параметры, отвечающие за ранее выученные задачи, замораживают. Такой приём подавляет катастрофическое забывание, однако, как и в случае с предыдущей группой, приводит к линейному росту объёма памяти и вычислительных издержек. Ранние методы, такие как PNN [25], имели простой принцип, интегрируя в архитектуру новые цепочки нейронов (нейронные сети единичной ширины), фиксируя веса предыдущих столбцов и обучая новые веса переиспользовать полезные признаки, сформировавшиеся в течение предыдущих тренировочных сессий. Такие методы имели типич-

ную для моделей с динамической архитектурой проблему с линейным ростом затрачиваемых ресурсов и требовали выбора правильных столбцов для получения предсказаний. Результатом дальнейших исследований стал гибридный метод Progress & Compress (P & C) [24], объединивший в себе методы PNN и EWC и добившийся работы с использованием постоянного объема памяти.

Так как используемые в классических алгоритмах с динамической архитектурой ансамблирование и расширение архитектуры не учитывают особенности структуры базовой модели, их адаптация для KAN не предоставит новой информации. В свою очередь, современные методы, например P & C, слишком сложны, чтобы их адаптировать. Поэтому было принято решение отказаться от дальнейшего рассмотрения методов из этой категории.

1.3 Подход с воспроизведением опыта

1.3.1 Повторение (Rehearsal)

Главный принцип методов с повторением заключается в сохранении небольшого подмножества элементов предыдущих тренировочных сессий и периодическом включении их в текущую выборку данных. Классическим примером, реализующим этот принцип является алгоритм Experience Replay (ER) [7]. Для повышения репрезентативности сохраняемых экземпляров, в случае инкрементального обучения классам, стремятся выбирать из тренировочных сессий такие примеры, которые наиболее близки к среднему представлению класса в признаковом пространстве. В других задачах непрерывного обучения отбор репрезентативных объектов может оказаться затруднителен.

Известным примером архитектуры с повторением является GeppNet [8], которая реализует отбор репрезентативных объектов через использование самоорганизующейся карты (self-organising map, SOM), выступающей в роли буфера долгосрочной памяти и организующая входные векторы на двумерной решётке с сохранением топологии исходного распределения.

Как правило, методы с повторением оказываются очень эффективны и сравнительно просты в реализации, однако их серьёзным недостатком является хранение реальных примеров, которое предполагает линейный рост объема необходимой памяти и может нарушать требования безопасности при

обработке конфиденциальных данных.

В данной работе, по причине высокой популярности и эффективности, методам с повторением отдавалось наибольшее предпочтение, в частности, благодаря существованию алгоритмов, совмещающих вышеописанные свойства с автоматическим формированием буфера памяти.

1.3.2 Псевдоповторение (Pseudo Rehearsal)

В основе идеи псевдоповторения лежит воспроизведение искусственных примеров из предыдущих тренировочных сессий, полученных с помощью генеративных моделей. Затем эти примеры объединяются с текущим набором данных в процессе обучения, что позволяет избежать явного хранения реальных образцов.

Искусственные примеры и соответствующие метки создаются посредством пропускания случайного шума через отдельную генеративную модель (например, GAN), но с ростом сложности генерация репрезентативных образцов может оказаться чрезвычайно трудной задачей. Такие псевдо-образцы часто не полностью отражают истинное распределение прошлых данных, поэтому отдельным направлением исследований является поиск способов сохранения способности генераторов синтезировать репрезентативные примеры. Кроме того, их качественное обучение и поддержка требуют значительных ресурсов и времени, что не всегда реализуемо на практике.

Таким образом, основное преимущество методов с псевдоповторением (pseudo rehearsal) перед методами с повторением (rehearsal) сводится к отсутствию необходимости хранения реальных данных, а значит, снижению требований к объему памяти и повышению безопасности при работе с чувствительными данными. Взамен появляется необходимость обучения отдельной генеративной нейронной сети, которая сама по себе будет подвержена катастрофическому забыванию и потребует использования регуляризации или других методов предотвращения появления смещений в её априорном распределении.

То, что генеративных нейронных сетей, используемых в составе методов с псевдоповторением, требуют больших вычислительных ресурсов, а также

то, что генератор сам подвержен забыванию, что усложняет эксперименты, стало причиной для отказа от их дальнейшего рассмотрения в рамках данного исследования.

1.3.3 С градиентными ограничениями (Constrained)

Методы с градиентными ограничениями также сохраняют в буфер подмножество примеров из предыдущих задач и используют их для вычисления градиента функции потерь на этих примерах при обучении на новой задаче. Если градиент в текущей тренировочной сессии конфликтует с градиентами буфера (то есть их скалярное произведение отрицательно), выполняется проекция нового градиента на выпуклую область, образованную этими градиентами, чтобы не избежать потери производительности в предыдущих задачах.

Наибольшую популярность среди constrained-методов приобрёл алгоритм Averaged Gradient Episodic Memory (A-GEM). В нём для ускорения вычислительной процедуры вместо градиентов функции потерь по каждому из содержащихся в буфере образцов используется усреднённый градиент буфера. Если градиент текущей задачи конфликтует с этим усреднённым градиентом, то градиент текущей задачи проецируется на гиперплоскость, ортогональную градиенту из буфера.

К преимуществам методов с градиентными ограничениями можно отнести архитектурную простоту и возможность выбора между формальными гарантиями невозрастания потерь на прошлых задачах (при вычислении градиентов по каждому из примеров в буфере памяти) и высокой производительности (при использовании упрощённых вариантов по типу A-GEM). Недостатки обусловлены главным образом наличием буфера памяти: требуется линейное по числу тренировочных сессий количество ресурсов, имеются проблемы с конфиденциальностью и необходимо решать, какие примеры хранить в памяти, чтобы содержание буфера было репрезентативным.

В целом, свойства методов из рассматриваемой категории схожи с таковыми у методов с повторением. Однако в отличие от них, методы с градиентными ограничениями разделены на две группы. Первая (вычисление всех градиентов в буфере памяти) сложна в реализации и вычислительно затратна,

а вторая (вычисление усреднённого градиента) из-за упрощённой процедуры вычисления градиентов может затруднить анализ результатов экспериментов. Обе эти группы недостаточно универсальны, так что окончательный выбор пал на методы с повторением.

Глава 2. Построение усовершенствованных моделей на базе архитектуры KAN

Для достижения целей, сформулированных в настоящем исследовании, в первую очередь потребовалась программная реализация нейронной сети Колмогорова–Арнольда. Как и MLP, KAN - это многослойная полносвязная нейронная сеть, состоящая из нейронов, граней и функций активации. Наглядное сравнение между этими архитектурами приведено на рисунке 1. Их главным отличие заключается в том, что в KAN функции активации перемещены с нейронов на грани и являются обучаемыми. Строение одного слоя KAN изображено на рисунке 4. Если обозначить вход слоя как $x \in \mathbb{R}^{n_x}$ и выход как $y \in \mathbb{R}^{n_y}$, где n_x и n_y - соответствующие размерности, то слой Φ можно задать матрицей функций:

$$\Phi = [\phi_{j,i}]_{j=1,\dots,n_y}^{i=1,\dots,n_x},$$

где каждая $\phi_{j,i} : \mathbb{R} \rightarrow \mathbb{R}$ - независимая обучаемая скалярная функция. Тогда вычисление выходных значений слоя - это просто суммирование:

$$y = \Phi \cdot x, \quad y_j = \sum_{i=1}^{n_x} \phi_{j,i}(x_i).$$

Существует множество способов параметризации функций активации $\phi_{j,i}$. Наибольший интерес для применения в непрерывном обучении представляют формулы со свойством локальности поддержки. Одним из таких вариантов является использование базисных сплайнов $B^{(k)}$ порядка k . Значение функции в данной точке вычисляется как линейная комбинация значений базисных сплайнов в этой точке с обучаемыми коэффициентами:

$$\phi_{j,i}(x) = \sum_{m=1}^k c_{j,i,m} B_m^{(k)}(x),$$

где $B_m^{(k)}$ - базисные функции на заранее заданной или адаптируемой сетке (см. рисунок 5), $c_{j,i,m}$ - обучаемые коэффициенты. В официальной программ-

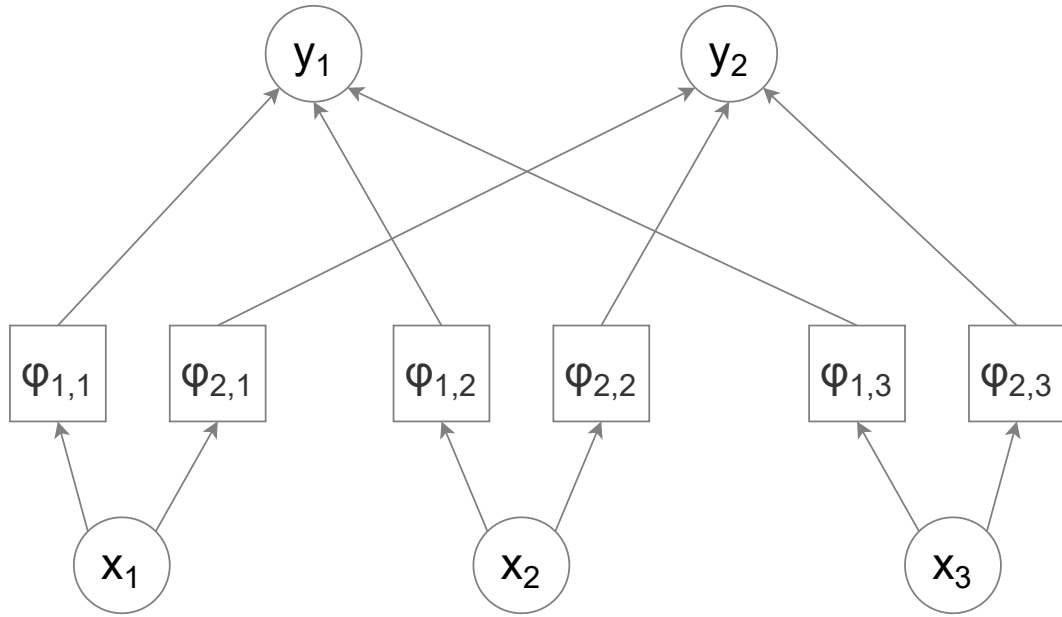


Рис. 4: Схематическое изображение одного слоя Φ нейронной сети Колмогорова-Арнольда

ной реализации KAN к значению сплайна в функции активации добавляют дополнительное слагаемое - нелинейную функцию b (по-умолчанию SiLU) и используют масштабирующие коэффициенты s_b и s_s :

$$\phi_{j,i}(x) = s_b b(x) + s_s \sum_{m=1}^k c_{j,i,m} B_m^{(k)}(x),$$

что по заявлениям разработчиков улучшает начальную стабильность обучения.

Из-за внесения этих изменений в архитектуру, официальная программная реализация KAN [18], как показано в [28], полностью утратила способности к сохранению информации в условиях непрерывного обучения и не могла быть использована в настоящем исследовании. По этой причине была разработана самостоятельная программная реализация сетей Колмогорова-Арнольда, имеющая ряд отличий от версии, предложенной в оригинальной научной публикации.

Ключевые отличия реализации

1. **В-сплайны фиксированного порядка.** Поскольку вычисление значения базисного сплайна произвольного порядка требует использования

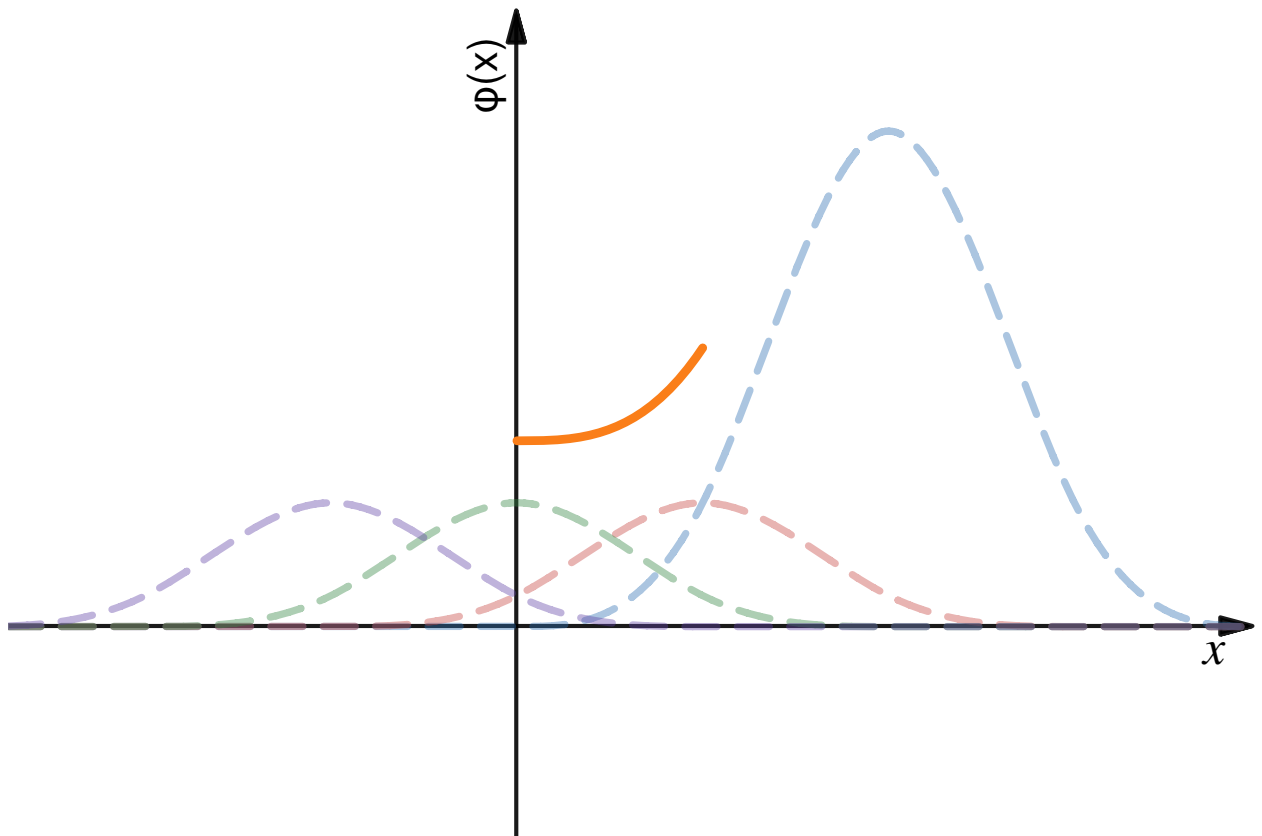


Рис. 5: Пример функции активации $\phi : [0, 1] \rightarrow \mathbb{R}$ (оранжевая сплошная кривая), параметризованной с помощью линейной комбинации четырех базисных сплайнов с коэффициентами 1,1,1,4 (разноцветные пунктирные кривые).

рекурсивного алгоритма Де Бура, было решено зафиксировать порядок используемых в функциях активации $\phi_{j,i}$ В-сплайнов. Благодаря этому удалось вывести явную формулу для кубического базисного сплайна и векторизовать операции вычисления его значения.

2. **Функций активации, состоящие только из линейной комбинации базисных сплайнов.** Масштабирующие коэффициенты s_b и s_s в условиях непрерывного обучения неизбежно оказываются общими параметрами для всех задач, так как изменяют значение функции активации во всех точках одновременно. Поэтому их оптимизация в текущей тренировочной сессии фактически обесценивает представления, содержащие в себе информацию о предыдущих задачах.
3. **Фиксированная сетка базисных функций вкупе, общая пакетная нормализация (Shared Batch Normalization) и ограничение области входных значений.** Как следует из [12], оригинальный подход, подра-

зумевающий адаптацию сетки базисных функций по мере появления новых объектов не может быть использован в непрерывном обучении. Вместо этого ко входным векторам применяется слой пакетной нормализации, а затем область входных значений ограничивается на отрезке $[0, 1]$.

4. **Процедура инициализации параметров**, адаптированная из [2]. Сначала для каждой функции активации в нейронной сети формируется равномерная сетка узлов G размером $|G|$. Затем для каждого узла генерируется целевое значение $B \sim N(0, \sigma^2)$, где $\sigma = \frac{1}{10(|G| - 3)}$. После этого коэффициенты базисных сплайнов (параметры модели) C вычисляются решением задачи наименьших квадратов $AC = B$, где A - начальные значения функций активации в узлах равномерной сетки. Такой подход обеспечивает интерполяцию функций активации через заданные в узлах значения

Исходный код программной реализации KAN, использованной в настоящем исследовании, приведён в приложении 1. Помимо этого, весь исходный код доступен в открытом репозитории [27].

2.1 Подход с использованием регуляризации

Среди методов противодействия катастрофическому забыванию, реализующих подход с использованием регуляризации, с целью адаптации для KAN был выбран EWC, который, согласно таксономии из главы 1, относится к подкатегории методов с весовой регуляризацией.

В KAN каждая обучаемая функция активации $\phi_{j,i}$ параметризуется коэффициентами кубических В-сплайнов $\{c_{j,i,m}\}_{m=1}^k$, обладающих локальной поддержкой. Следовательно, изменение отдельного коэффициента влияет лишь на ограниченный диапазон входных значений, что уменьшает перекрытие между представлениями, относящимся к разным задачам. Регуляризация $\theta = \{c_{j,i,m}\}$ должна усилить локальный характер обновлений и подавить избыточные глобальные сдвиги, приводящие к катастрофическому забыванию в оригинальной реализации KAN.

Основная идея EWC заключается во включении в стандартную функцию потерь $\mathcal{L}$ дополнительного регуляризирующего слагаемого, штрафующего отклонение параметров θ от значений $\theta^{(t-1)}$, полученных после предыдущей тренировочной сессии $t - 1$:

$$\mathcal{L}_{\text{EWC}}(f_{\theta}(x), y) = \mathcal{L}(f_{\theta}(x), y) + \frac{\lambda}{2} \sum_i F_i^{(t-1)} (\theta_i - \theta_i^*)^2,$$

где λ — гиперпараметр, определяющий силу регуляризации, а $F^{(t-1)}$ — диагональная аппроксимация информационной матрицы Фишера, служащая оценкой важности каждого параметра для задач, выученных ранее. В классической постановке для каждой задачи хранится собственная θ^* и своя F , но такое решение требует большого объёма памяти, что непрактично при большом числе сессий.

Поэтому в настоящей работе реализована модификация оригинального метода под названием online EWC [24], где используется единая матрица Фишера $F^{(t)}$ для всех задач, обновляемая по правилу экспоненциального сглаживания:

$$F_i^{(t)} = \gamma F_i^{(t-1)} + (1 - \gamma) \hat{F}_i^{(t)}, \quad 0 < \gamma < 1,$$

где $\hat{F}^{(t)}$ — свежая оценка Фишера на данных текущей сессии, а γ — коэффициент затухания. Благодаря этому объём дополнительной памяти становится фиксированным, но регуляризация постепенно «забывает» статистику давно прошедших задач.

Расчёт оценки матрицы Фишера $\hat{F}^{(t)}$ производится на данных текущей тренировочной сессии B_t размером N_t :

$$\hat{F}_i^{(t)} = \frac{1}{N_t} \sum_{(x,y) \in B_t} \left(\frac{\partial \mathcal{L}}{\partial \theta_i} \Big|_{(x,y)} \right)^2,$$

по-умолчанию реализовано взятие градиента именно по модельным предсказаниям (expected Fisher), а не по истинным меткам классов (empirical Fisher). В программной реализации присутствуют обе версии, но в статье [14] исполь-

зование эмпирической матрицы Фишера было подвержено критике.

Ключевые отличия реализации

1. **Онлайн-обновление.** Вместо хранения набора пар (θ^*, F) для всех завершённых задач поддерживается единственный вектор $F^{(t)}$ и набор параметров θ^* , перезаписываемые после каждой тренировочной сессии. Это снижает требование к памяти и ускоряет доступ к статистике.
2. **Адаптивная оценка меток.** Флаг `empirical_estimate` позволяет подменять истинные метки y предсказаниями модели $\arg \max f_{\theta}(x_b)$, что позволяет выбирать между вычислением эмпирической и ожидаемой матрицами Фишера.

Программный код функции потерь `EWCLoss` приведён в приложении 3. Он реализует все вышеперечисленные модификации и может быть использован как на месте обычной функции потерь из `PyTorch`.

2.2 Подход с изменением архитектуры

Среди методов противодействия катастрофическому забыванию, реализующих подход с изменением архитектуры, с целью адаптации для KAN был выбран FEL, который, согласно таксономии из главы 1, относится к подкатегории методов с фиксированной архитектурой.

2.3 Подход с воспроизведением опыта

Среди методов противодействия катастрофическому забыванию, реализующих подход с воспроизведением опыта, с целью адаптации для KAN был выбран GeppNet, который, согласно таксономии из главы 1, относится к подкатегории методов с повторением.

В основе алгоритма GeppNet лежит самоорганизующаяся карта (Self-Organizing Map, SOM), служащая «буфером долговременной памяти», — она проецирует входные векторы $x \in \mathbb{R}^d$ на двумерную решётку прототипов $W = \{w_j \in \mathbb{R}^d\}_{j=1}^M$ и тем самым автоматически структурирует поток входных

данных без сохранения отдельных примеров. Для получения предсказаний активность каждого прототипа рассчитывается как

$$s_j(x) = \exp\left(-\frac{\|x-w_j\|^2}{2\sigma_{\text{in}}^2}\right), \quad j = 1, \dots, M,$$

где σ_{in} — дисперсия в пространстве признаков. Полученный вектор $s(x) \in \mathbb{R}^M$ используется в качестве входных данных для следующего слоя. В классическом GeppNet это MLP, а в настоящей работе - слой KAN.

В оригинальном GeppNet прототипы обновляются после каждой выборки в режиме online-обучения, что делает невозможным оптимизацию вычислительных процедур, поэтому вместо обычной SOM была реализована её модификация - Batch-SOM [4]. Для батча $X = \{x_i\}_{i=1}^B$ ищутся ближайшие прототипы в пространстве признаков (best matching units, BMU):

$$\text{bmu}(x) = \arg \min_j \|x - w_j\|,$$

после чего происходит обновлением прототипов:

$$w_j = \frac{\sum_{i=1}^B h_{j,\text{bmu}(x_i)} x_i}{\sum_{i=1}^B h_{j,\text{bmu}(x_i)}}, \quad h_{j,k} = \exp\left(-\frac{\|c_j - c_k\|^2}{2\sigma_{\text{grid}}^2}\right),$$

где $c_j \in \mathbb{R}^2$ — координаты прототипа w_j на квадратной решётке, а σ_{grid} — дисперсия в пространстве координат.

Ключевые отличия реализации

1. **Инициализация прототипов.** Перед началом применения SOM проходит через одну эпоху обучения на базовом подмножестве данных, чтобы адаптировать под данные стартовое расположение W , а также ускорить дальнейшее упорядочивание прототипов.
2. **Раздельное обучение SOM и KAN.** Поскольку слой KAN принимает градиенты от $\mathcal{L}$, а SOM обновляется отдельно, процедура обучения чередует оптимизацию KAN-классификатора и обновление прототипов, что не требует изменения стандартного алгоритма обучения.

Код для реализованных с помощью PyTorch класса SOMLayer, а также использующего его совместно с KAN класса RehearsalKAN находится в приложении 4.

Глава 3. Применение усовершенствованных моделей в задачах непрерывного обучения

CUB-200-2011	
Объект выборки	RGB изображение
Количество классов	200
Количество признаков	2,048
Размер обучающей выборки	5,994
Размер тестовой выборки	5,794
Тренировочные объекты/класс	29-30
Тестовые объекты/класс	11-30

Таблица 1: Описание используемого набора данных

В качестве набора данных используется CUB-200-2011 (Таблица 1), содержащий в себе привычное для прикладных задач количество объектов и классов. Поскольку основная цель исследования - оценка эффективности противодействия моделей катастрофическому забыванию, а не сравнение качества векторных представлений объектов обучающей выборки, в качестве входных признаков используются векторные представления, полученные из предпоследнего слоя предобученной модели ResNet-50.

Согласно общепринятым рекомендациям, проводится несколько экспериментов:

1. Обучение с перестановкой признаков.
2. Инкрементальное обучение классам.

Первый эксперимент оценивает способность модели сохранять несколько представлений одного и того же набора данных, обучаясь на них последовательно. Второй эксперимент направлен на изучение способности модели обучаться новым классам данных, не забывая при этом ранее изученные классы.

3.1 Обучение с перестановкой признаков

3.2 Инкрементальное обучение классам

Глава 4. Анализ экспериментов

Выводы

Заключение

Список литературы

1. A Comprehensive Survey of Continual Learning: Theory, Method and Application / L. Wang [и др.] // arXiv preprint arXiv:2302.00487. — 2024. — DOI: 10.48550/arXiv.2302.00487. — arXiv: 2302.00487. — URL: <https://arxiv.org/abs/2302.00487> ; Accepted (concise version) in IEEE TPAMI.
2. *Blealtan*. Efficient-KAN: An Efficient Implementation of Kolmogorov-Arnold Network. — 2024. — URL: <https://github.com/Blealtan/efficient-kan> (дата обр. 15.05.2025). GitHub repository.
3. *Chen Z., Liu B.* Lifelong Machine Learning. — Cham : Springer, 2017. — (Synthesis Lectures on Artificial Intelligence and Machine Learning). — ISBN 978-3-031-01575-5. — DOI: 10.1007/978-3-031-01575-5.
4. *Cheng Y.* Convergence and Ordering of Kohonen's Batch Map // Neural Computation. — 1997. — Т. 9, № 8. — С. 1667—1676. — DOI: 10.1162/neco.1997.9.8.1667. — URL: <https://doi.org/10.1162/neco.1997.9.8.1667> (дата обр. 06.05.2025).
5. Continual Lifelong Learning with Neural Networks: A Review / G. I. Parisi [и др.]. — 2018. — DOI: 10.48550/arXiv.1802.07569. — arXiv: 1802.07569 [cs.LG]. — URL: <https://arxiv.org/abs/1802.07569> (дата обр. 06.05.2025) ; version 4, revised 2019-02-11.
6. *Coop R., Mishtal A., Arel I.* Ensemble Learning in Fixed Expansion Layer Networks for Mitigating Catastrophic Forgetting // IEEE Transactions on Neural Networks and Learning Systems. — 2013. — Т. 24, № 10. — С. 1623—1634. — DOI: 10.1109/TNNLS.2013.2264952.
7. Experience Replay for Continual Learning / D. Rolnick [и др.] // Advances in Neural Information Processing Systems. Т. 32 / под ред. Н. М. Wallach [и др.]. — Curran Associates, Inc., 2019. — С. 348—358. — (Proceedings of Neural Information Processing Systems). — DOI: 10.48550/arXiv.1811.11682. — URL: <https://papers.nips.cc/paper/8327-experience-replay-for-continual-learning> (дата обр. 13.05.2025).

8. *Gepperth A., Karaoguz C.* A Bio-Inspired Incremental Learning Architecture for Applied Perceptual Problems // *Cognitive Computation*. — 2016. — Т. 8, № 5. — С. 924—934. — DOI: 10.1007/s12559-016-9389-5.
9. *Hinton G., Vinyals O., Dean J.* Distilling the Knowledge in a Neural Network. — 2015. — arXiv: 1503.02531 [stat.ML]. — URL: <https://arxiv.org/abs/1503.02531> (дата обр. 11.05.2025) ; version 1.
10. KAN 2.0: Kolmogorov-Arnold Networks Meet Science / Z. Liu [и др.]. — 2024. — DOI: 10.48550/arXiv.2408.10205. — arXiv: 2408.10205 [cs.LG]. — URL: <https://arxiv.org/abs/2408.10205> (дата обр. 05.05.2025) ; version 1.
11. KAN: Kolmogorov-Arnold Networks / Z. Liu [и др.]. — 2024. — DOI: 10.48550/arXiv.2404.19756. — arXiv: 2404.19756 [cs.LG]. — URL: <https://arxiv.org/abs/2404.19756> (дата обр. 02.05.2025) ; Accepted at ICLR 2025, version 5 (2025-02-09).
12. Kolmogorov-Arnold Networks Still Catastrophically Forget but Differently from MLP / A. Lee [и др.] // *Proceedings of the AAAI Conference on Artificial Intelligence*. Т. 39. — 2025. — С. 18053—18061. — DOI: 10.1609/aaai.v39i17.33986. — URL: <https://ojs.aaai.org/index.php/AAAI/article/view/33986> ; Technical Track on Machine Learning III.
13. *Kumar A., Agarwal S., Hemanth D. J.* A Methodology-Oriented Study of Catastrophic Forgetting in Incremental Deep Neural Networks. — 2024. — DOI: 10.48550/arXiv.2405.08015. — arXiv: 2405.08015 [cs.LG]. — URL: <https://arxiv.org/abs/2405.08015> (дата обр. 10.05.2025) ; version 1.
14. *Kunstner F., Balles L., Hennig P.* Limitations of the Empirical Fisher Approximation for Natural Gradient Descent. — 2019. — arXiv: 1905.12558 [cs.LG]. — URL: <https://arxiv.org/abs/1905.12558> (дата обр. 15.05.2025) ; Version 3: Minor corrections (typographic errors).

15. Learn++: An Incremental Learning Algorithm for Supervised Neural Networks / R. Polikar [и др.] // IEEE Transactions on Systems, Man and Cybernetics, Part C: Applications and Reviews. — 2001. — Т. 31, № 4. — С. 497—508. — DOI: 10.1109/5326.983933. — URL: <https://doi.org/10.1109/5326.983933> (дата обр. 13.05.2025).
16. Li Z., Hoiem D. Learning without Forgetting // Computer Vision - 14th European Conference on Computer Vision, ECCV 2016, Proceedings. Т. 9908 / под ред. B. Leibe [и др.]. — Cham : Springer, 2016. — С. 614—629. — (Lecture Notes in Computer Science). — DOI: 10.1007/978-3-319-46493-0_37. — URL: https://doi.org/10.1007/978-3-319-46493-0_37 (дата обр. 11.05.2025).
17. Life-long Learning Based on Dynamic Combination Model / B. Ren [и др.] // Applied Soft Computing. — 2017. — Т. 56. — С. 398—404. — DOI: 10.1016/j.asoc.2017.03.005. — URL: <https://doi.org/10.1016/j.asoc.2017.03.005> (дата обр. 13.05.2025).
18. Liu Z. PyKAN: Kolmogorov-Arnold Networks. — 2024. — URL: <https://github.com/KindXiaoming/pykan> (дата обр. 06.05.2025). GitHub repository.
19. Machine Learning for Data Streams: with Practical Examples in MOA / A. Bifet [и др.]. — Cambridge, MA : MIT Press, 2018. — (Adaptive Computation and Machine Learning). — ISBN 978-0262037792. — DOI: 10.7551/mitpress/10654.001.0001. — URL: <https://moa.cms.waikato.ac.nz/book/> (дата обр. 06.05.2025) ; 1st edition.
20. Measuring Catastrophic Forgetting in Neural Networks / R. Kemker [и др.]. — 2017. — DOI: 10.48550/arXiv.1708.02072. — arXiv: 1708.02072 [cs.AI]. — URL: <https://arxiv.org/abs/1708.02072> (дата обр. 02.05.2025) ; To appear in AAAI 2018.
21. Memory Aware Synapses: Learning what (not) to forget / R. Aljundi [и др.] // Proceedings of the European Conference on Computer Vision (ECCV). — 2018. — URL: <https://arxiv.org/abs/1711.09601> (дата обр. 11.05.2025).

22. Overcoming Catastrophic Forgetting in Neural Networks / J. Kirkpatrick [и др.] // Proceedings of the National Academy of Sciences. — 2017. — Т. 114, № 13. — С. 3521—3526. — DOI: 10.1073/pnas.1611835114.
23. PathNet: Evolution Channels Gradient Descent in Super Neural Networks / C. Fernando [и др.]. — 2017. — DOI: 10.48550/arXiv.1701.08734. — arXiv: 1701.08734 [cs.NE]. — URL: <https://arxiv.org/abs/1701.08734> (дата обр. 02.05.2025) ; version 3.
24. Progress & Compress: A Scalable Framework for Continual Learning / J. Schwarz [и др.] // Proceedings of the 35th International Conference on Machine Learning. — 2018. — С. 4528—4537. — URL: <https://proceedings.mlr.press/v80/schwarz18a.html> (дата обр. 06.05.2025).
25. Progressive Neural Networks / A. A. Rusu [и др.]. — 2016. — DOI: 10.48550/arXiv.1606.04671. — arXiv: 1606.04671 [cs.LG]. — URL: <https://arxiv.org/abs/1606.04671> (дата обр. 13.05.2025) ; version 4.
26. Subspace Distillation for Continual Learning / K. Roy [и др.] // Neural Networks. — 2023. — Т. 167. — С. 65—79. — DOI: 10.1016/j.neunet.2023.07.047. — URL: <https://doi.org/10.1016/j.neunet.2023.07.047> (дата обр. 11.05.2025).
27. *Tretyakov D.* SPBU Bachelor Thesis. — 2025. — URL: <https://github.com/trxxxxkov/spbu-bachelor-thesis> (дата обр. 02.05.2025). GitHub repository.
28. *Yu R., Yu W., Wang X.* KAN or MLP: A Fairer Comparison. — 2024. — DOI: 10.48550/arXiv.2407.16674. — arXiv: 2407.16674 [cs.LG]. — URL: <https://arxiv.org/abs/2407.16674> (дата обр. 05.05.2025) ; version 2.
29. *Zenke F., Poole B., Ganguli S.* Continual Learning Through Synaptic Intelligence // Proceedings of the 34th International Conference on Machine Learning. — 2017. — С. 3987—3995. — URL: <https://proceedings.mlr.press/v70/zenke17a.html> (дата обр. 11.05.2025).

Приложения

Глава А. Исходный код усовершенствованных нейронных сетей

Приложение 1: Исходный код на Python для модели на основе архитектуры KAN

```
1 """Implementation of a KAN layer and a baseline KAN model that are used in the
2 CL experiments."""
3
4 import math
5
6 import torch
7 import torch.nn.functional as F
8
9
10 class KANLayer(torch.nn.Module):
11     """Highly optimized KAN layer that parametrizes the activation functions with
12     basis splines of fixed order.
13
14     Key differences from other implementations:
15     1. B-splines has a fixed order of 4 (polynomial functions of degree 3).
16     2. Output is a B-splines linear combination only (no classic activation
17     function is added and no scaling is performed).
18     3. Input range is fixed (and so is B-spline grid): [0, 1];
19     4. Has a shared batch normalization layer built-in;
20     5. Uses weight initialisation taken from https://github.com/Blealtan/efficient-kan
21     """
22
23     spline_order = 3
24
25     def __init__(
26         self,
27         in_features: int,
28         out_features: int,
29         grid_size: int = 3,
30     ):
31         super().__init__()
32         self.in_features = in_features
33         self.out_features = out_features
34         self.grid_step = 1 / grid_size
35         self.bn = torch.nn.BatchNorm1d(in_features)
36         knots_num = math.ceil(1 / self.grid_step) + 1 + self.spline_order
37         # Coefficients for basis functions (aka control points)
38         self.bspline_coeffs = torch.nn.Parameter(
39             torch.Tensor(out_features, in_features, knots_num)
```

```

40     )
41     grid = (
42         torch.arange(
43             -self.spline_order, knots_num - self.spline_order, dtype=torch.float
44         )
45         * self.grid_step
46     ).contiguous()
47     # Explicit formulas for cubic B-spline on the four intervals:
48     # B(s)=s^3/6, s in [0,1)
49     # B(s)=(-3s^3+12s^2-12s+4)/6, s in [1,2)
50     # B(s)=(3s^3-24s^2+60s-44)/6, s in [2,3)
51     # B(s)=(-s^3+12s^2-48s+64)/6, s in [3,4)
52     cubic_bspline_formula = torch.tensor(
53         [
54             [1 / 6, 0, 0, 0],
55             [-3 / 6, 12 / 6, -12 / 6, 4 / 6],
56             [3 / 6, -24 / 6, 60 / 6, -44 / 6],
57             [-1 / 6, 12 / 6, -48 / 6, 64 / 6],
58         ],
59         dtype=torch.float,
60     ).contiguous()
61     self.register_buffer("grid", grid)
62     self.register_buffer("cubic_bspline_formula", cubic_bspline_formula)
63     self._initialize_params()
64
65     def forward(self, x: torch.Tensor) -> torch.Tensor:
66         """Calculate output values as a linear combination of B-splines(x)"""
67         # Normalize and clip input vales to fit in the fixed spline grids' range
68         x = torch.clip(self.bn(x), self.grid[self.spline_order], self.grid[-1])
69         output = F.linear( # pylint: disable=E1102
70             self._bsplines_values_at(x).view(x.shape[0], -1),
71             self.bspline_coeffs.view(self.out_features, -1),
72         )
73         return output
74
75     def _initialize_params(self) -> None:
76         """Initialize B-splines' coefficients by interpolating uniform noise.
77
78         A least squares problem is considered for Ax = B, where
79         x - B-splines' values computed at grid knots (4 per each value in x),
80         A - optimal control points, that interpolate reference values,
81         B - reference values sampled from a centered scaled normal distribution.
82         """
83         with torch.no_grad():
84             # Get knots inside the grid range
85             interpolation_nodes = self.grid.expand(self.in_features, -1).T[
86                 self.spline_order :
87                 ]

```



```

88     # B-splines' values for the chosen points. This is "x" in the system
89     # of linear equations mentioned in the method's docstring.
90     current_values = self._bsplines_values_at(interpolation_nodes).transpose(
91         0, 1
92     )
93     # Spline values to be interpolated by adjusting coefficients of
94     # B-splines considering current_values. This is "B" in the
95     # Ax = B system of linear equations.
96     reference_values = (
97         torch.randn(
98             self.in_features,
99             len(self.grid) - self.spline_order,
100            self.out_features,
101        )
102    ) / (10 * (len(self.grid) - self.spline_order))
103    # Optimal B-splines' coefficients that interpolate the reference
104    # values
105    coefficients = torch.linalg.lstsq( # pylint: disable=E1102
106        current_values, reference_values
107    ).solution
108    self.bspline_coeffs.data.copy_(coefficients.permute(2, 0, 1))
109
110    def _bsplines_values_at(self, x: torch.Tensor) -> torch.Tensor:
111        """Compute values of all cubic B-splines at input point x.
112
113        Normalize x to grid scale, find spline_order+1 B-splines per input that
114        defines the output value and compute B-splines at those positions.
115
116        Args:
117        x: Input tensor with values in [0,1] range
118
119        Returns:
120        bsplines_values: Tensor of shape [batch_size, in_features, len(grid)]
121        where each element contains the evaluated i-th left B-splines
122        at point x.
123        """
124        x_norm = (x - self.grid[0]) / self.grid_step
125        closest_left_knot = torch.floor(x_norm).long()
126        bsplines_values = torch.zeros((*x.shape, len(self.grid)), device=x.device)
127        for i in range(4):
128            # Retrieve coefficients for the explicit formula of a B-spline for the
129            # current interval
130            coeff_row = self.cubic_bspline_formula[3 - i]
131            ith_left_knot = closest_left_knot - 3 + i
132            # Calculate input value for the i-th left knot considering it's distance
133            ith_left_bspline_arg = (3 - i) + (x_norm - closest_left_knot)
134            ith_left_bspline_value = (
135                coeff_row[0] * ith_left_bspline_arg.pow(3)

```

```

136         + coeff_row[1] * ith_left_bspline_arg.pow(2)
137         + coeff_row[2] * ith_left_bspline_arg
138         + coeff_row[3]
139     )
140     bsplines_values.scatter_add_(
141         dim=2,
142         index=ith_left_knot.unsqueeze(-1),
143         src=ith_left_bspline_value.unsqueeze(-1),
144     )
145     return bsplines_values.contiguous()
146
147
148 class BaselineKAN(torch.nn.Module):
149     """Kolmogorov-Arnold Network model used as a reference for performance comparison
150
151     This model is trained offline and serves as a baseline for evaluating the
152     performance of proposed models trained in an online learning setting.
153     """
154
155     def __init__(
156         self,
157         input_dim: int = 2048,
158         hidden_dim: int = 400,
159         output_dim: int = 200,
160     ):
161         super().__init__()
162         self.classifier = torch.nn.Sequential(
163             KANLayer(input_dim, hidden_dim), KANLayer(hidden_dim, output_dim)
164         )
165
166     def forward(self, x: torch.Tensor):
167         return self.classifier(x)

```

Приложение 2: Исходный код на Python для усовершенствованной модели, реализующей подход с изменением архитектуры

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 class FELayer(nn.Module):
7     """
8     Fixed Expansion Layer (FEL) - модульный разреженный слой для PyTorch.
9     Параметры
10     -----
11     in_features : int - размер входа

```

```

12 out_features : int - размер расширенного пространства (>> in_features)
13 k_pos       : int - сколько нейронов с макс. активацией оставить
14 k_neg       : int - сколько нейронов с мин. активацией оставить
15 excitatory_ratio : float - доля возбуждающих связей для каждого FEL-нейрона
16 """
17
18 def __init__(
19     self,
20     in_features: int,
21     out_features: int,
22     k_pos: int = 25,
23     k_neg: int = 25,
24     excitatory_ratio: float = 0.5,
25     weight_scale: float = 1.0,
26 ):
27     super().__init__()
28     self.in_features = in_features
29     self.out_features = out_features
30     self.k_pos = k_pos
31     self.k_neg = k_neg
32
33     # --- разрежённая фиксированная матрица W ---
34     # +1 / -1 с одинаковым числом входов на нейрон
35     conn_per_neuron = max(1, int(excitatory_ratio * in_features))
36     mask = torch.zeros(out_features, in_features, dtype=torch.float32)
37     for j in range(out_features):
38         idx = torch.randperm(in_features)[:conn_per_neuron]
39         half = conn_per_neuron // 2
40         mask[j, idx[:half]] = weight_scale # возбуждающие
41         mask[j, idx[half:]] = -weight_scale # ингибирующие
42     self.register_buffer("W_fixed", mask) # градиент не требуется
43
44 def forward(self, x: torch.Tensor) -> torch.Tensor:
45     """
46     x: (batch, in_features)
47     out: (batch, out_features) - триггерованная разрежённая активация
48     """
49     # линейное преобразование (нет обучения весов)
50     z = F.linear(x, self.W_fixed) # (B, out_features) # pylint: disable=E1102
51
52     # --- триггерование: top-K+, bottom-K- ---
53     with torch.no_grad():
54         # индексы нейронов с макс/мин активациями
55         topk_pos = torch.topk(z, self.k_pos, dim=1, largest=True).indices
56         topk_neg = torch.topk(z, self.k_neg, dim=1, largest=False).indices
57
58         trigger_mask = torch.zeros_like(z, dtype=torch.bool)
59         trigger_mask.scatter_(1, topk_pos, True)

```

```

60         trigger_mask.scatter_(1, topk_neg, True)
61
62         # фиксированные значения для активных узлов;
63         # detach() предотвращает градиент через константы
64         pos_val = z.max(dim=1, keepdim=True).values.detach()
65         neg_val = z.min(dim=1, keepdim=True).values.detach()
66
67         out = torch.zeros_like(z)
68         out[trigger_mask & (z >= 0)] = pos_val.expand(-1, self.out_features)[
69             trigger_mask & (z >= 0)
70         ]
71         out[trigger_mask & (z < 0)] = neg_val.expand(-1, self.out_features)[
72             trigger_mask & (z < 0)
73         ]
74
75         # в остальном - нули, градиент течёт через out к предыдущим слоям
76         return out

```

Приложение 3: Исходный код на Python для усовершенствованной модели, реализующей подход с использованием регуляризации

```

1  """Implementation of a custom loss function (based on EWC approach) which is a part
2  of regularized KAN"""
3
4  import torch
5  from torch.utils.data import DataLoader
6
7
8  class EWCLoss(torch.nn.Module):
9      """A loss function implementing Elastic Weight Consolidation (EWC) approach
10     using diagonal Fisher Information Matrix (FIM) to approximate posterior
11     distribution for previous tasks.
12
13     Key components:
14     1. Fisher Information Matrix: Stored in self.fisher and updated with
15     information from each new study session using exponential moving
16     average (decay factor stored in self.decay).
17     2. Previous model parameters: Stored in self.saved_params and overwritten
18     during each study session (Online EWC variant).
19
20     Regularization is not applied during the first study session.
21
22     To update FIM and save current model parameters, call the update() method
23     after each study session."""
24
25     def __init__(
26         self,

```

```

27     base_loss: torch.nn.Module,
28     model: torch.nn.Module,
29     lambda_: float = 1e6,
30     decay: float = 0.95,
31     device: torch.device = torch.device("cpu"),
32 ):
33     super().__init__()
34     self.base_loss = base_loss
35     self.model = model
36     self.lambda_ = lambda_
37     self.decay = decay
38     self.device = device
39     # Diagonal Fisher Information Matrix
40     self.fisher: dict = None
41     # Model parameters from the previous study session
42     self.saved_params: dict = None
43
44 def update(
45     self,
46     dataset: torch.utils.data.Dataset,
47     batch_size: int = 256,
48     num_batches: int = 200,
49     empirical_estimate: bool = False,
50     device: torch.device = torch.device("cpu"),
51 ) -> None:
52     """Determines current model's weights importance by updating Fisher
53     Information Matrix and saving current model's parameters.
54
55     Args:
56         dataset: Training data for FIM estimation
57         batch_size: Batch size for FIM calculation
58         num_batches: Number of batches to process
59         empirical_estimate: Use model predictions instead of true labels
60         device: Computation device
61     """
62     self.model.to(device)
63     self.model.eval()
64     # FIM is updated using exponential moving average:
65     # [current FIM] = self.decay * [old FIM] + (1-self.decay) * [new FIM]
66     new_fisher = {}
67     dataloader = DataLoader(
68         dataset, batch_size=batch_size, shuffle=True, num_workers=2
69     )
70     for batch_idx, (batch_inputs, batch_targets) in enumerate(dataloader):
71         # Only a part of the dataset is processed for optimization reasons
72         if batch_idx == num_batches:
73             break
74         batch_inputs = batch_inputs.to(device)

```

```

75     batch_targets = batch_targets.to(device)
76     batch_outputs = self.model(batch_inputs)
77     # See https://arxiv.org/pdf/1905.12558 for more information on the
78     # choice between FIM and empirical FIM
79     labels = (
80         batch_outputs.argmax(dim=1) if empirical_estimate else batch_targets
81     )
82     self.model.zero_grad()
83     batch_loss = self.base_loss(batch_outputs, labels)
84     batch_loss.backward()
85     for name, param in self.model.named_parameters():
86         if param.grad is not None:
87             grad_squared = param.grad.detach().clone().pow(2)
88             if name in new_fisher:
89                 new_fisher[name] += grad_squared
90             else:
91                 new_fisher[name] = grad_squared
92     # Average and update a FIM
93     for name in new_fisher:
94         new_fisher[name] /= num_batches
95     if self.fisher:
96         # EMA if there is a FIM calculated during previous tasks
97         for name, new_val in new_fisher.items():
98             old_val = self.fisher.get(name, torch.zeros_like(new_val))
99             self.fisher[name] = self.decay * old_val + (1 - self.decay) * new_val
100     else:
101         # Just a current FIM if it's the first study session
102         self.fisher = new_fisher
103     # There is only one set of model's parameters which is overwritten at each
104     # study session in contrast with the original EWC approach
105     self.saved_params = {
106         name: param.detach().clone()
107         for name, param in self.model.named_parameters()
108     }
109     self.model.train()
110
111 def forward(self, preds, targets, *args, **kwargs):
112     """Computes total loss: base_loss + lambda/2 * EWC regularization."""
113     base_loss = self.base_loss(preds, targets, *args, **kwargs)
114     ewc_loss = torch.tensor(0, dtype=float, device=preds.device)
115     # Regularization is not applied during the first study session
116     if self.fisher:
117         for name, param in self.model.named_parameters():
118             if param.requires_grad and (name in self.fisher):
119                 ewc_loss += (
120                     self.fisher[name] * (param - self.saved_params[name]).pow(2)
121                 ).sum()
122     return base_loss + (self.lambda_ / 2) * ewc_loss

```

Приложение 4: Исходный код на Python для усовершенствованной модели, реализующей подход с воспроизведением опыта

```
1  """Implementation of a rehearsal-based KAN model and a self-organizing maps-based
2  layer it relies on"""
3
4  import torch
5  from tqdm import tqdm
6
7  from spbu_bachelor_thesis.nn.kan import KANLayer
8
9
10 class SOMLayer(torch.nn.Module):
11     """Self-Organizing Map layer used in the Rehearsal KAN architecture. The map
12     has a square topology and is updated manually, once per training epoch, by
13     calling an update() method."""
14
15     def __init__(
16         self,
17         in_features: int,
18         out_features: int,
19         variance_init: float = 0.5,
20         variance_lr: float = 0.05,
21     ):
22         # A square grid topology is used for the self-organizing map and
23         # out_features is equal to a number of prototypes in it
24         if int(out_features**0.5) ** 2 != out_features:
25             raise ValueError("out_features must be a perfect square")
26         super().__init__()
27
28         self.fl = torch.nn.Flatten()
29
30         self.side = int(out_features**0.5)
31         self.var_lr = variance_lr
32         self.var_init = variance_init
33         self.weight: torch.Tensor
34         self.register_buffer("weight", torch.rand(out_features, in_features))
35         idx = torch.arange(self.side).expand(self.side, -1)
36         # Tensor with grid coordinates for each prototype
37         self.register_buffer(
38             "coords",
39             torch.stack([idx.T.reshape(-1), idx.reshape(-1)], dim=1).float(),
40         )
41         self.register_buffer("input_variance", torch.tensor(self.var_init))
42         self.register_buffer("grid_variance", torch.tensor(self.var_init * self.side))
43
44     def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```

45         """Return neural activity tensor"""
46
47         x = self.fl(x)
48
49         similarities = self._get_input_space_similarities(x)
50         if self.training:
51             self.update(x)
52         else:
53             self.grid_variance.copy_(self.var_init * self.side)
54             # sparse_similarities = self._sparsify_similarities(similarities)
55             # return sparse_similarities
56         return similarities
57
58     @torch.no_grad()
59     def update(self, x: torch.Tensor) -> None:
60         """Move prototypes toward corresponding BMUs and update variances"""
61         x = self.fl(x)
62         input_dist = torch.cdist(x, self.weight)
63         # Get indices for the best matching units (nearest prototypes to
64         # each of the input vectors)
65         bmu_idx = input_dist.argmax(dim=1)
66         grid_similarities = self._get_grid_similarities(bmu_idx)
67         numerator = grid_similarities.T @ x
68         denominator = grid_similarities.sum(dim=0)
69         # Batch-SOM update rule: sum(similarities * x) / sum(similarities)
70         self.weight.copy_((numerator / denominator.unsqueeze(1)).nan_to_num(0.0))
71         self.input_variance.copy_(
72             (1 - self.var_lr) * self.input_variance + self.var_lr * input_dist.mean()
73         )
74         self.grid_variance.copy_((1 - self.var_lr) * self.grid_variance)
75
76         # def _sparsify_similarities(self, similarities: torch.Tensor) -> torch.Tensor:
77         #     """Sparsify similarities by sharpening it and comparing with a threshold"""
78         #     sharp_similarities = similarities.pow(self.sharp_order) / (
79         #         similarities.max(dim=1, keepdim=True).values + 1e-6
80         #     ).pow(self.sharp_order - 1)
81         #     sparse_mask = (sharp_similarities >= self.sparse_min).float()
82         #     return sharp_similarities * sparse_mask
83
84     def _get_grid_similarities(self, bmu_idx: torch.Tensor) -> torch.Tensor:
85         """Calculate a grid similarity between given best matching units and
86         other prototypes."""
87         bmu_coords = self.coords[bmu_idx]
88         grid_distances = torch.cdist(bmu_coords, self.coords).to(bmu_idx.device)
89         grid_similarities = _gaussian(grid_distances, self.grid_variance)
90         return grid_similarities
91
92     def _get_input_space_similarities(self, x: torch.Tensor) -> torch.Tensor:

```



```

93         """Measure similarity between an input tensor and self.prototypes"""
94         input_space_distances = torch.cdist(x, self.weight)
95         return _gaussian(input_space_distances, self.input_variance)
96
97
98 def _gaussian(x: torch.Tensor, variance: torch.Tensor) -> torch.Tensor:
99     """Calculate zero-mean Gauss function with a given variance"""
100     return torch.exp(-x.pow(2) / (2 * variance.clamp(min=1e-6) ** 2)).to(x.device)
101
102
103 class RehearsalKAN(torch.nn.Module):
104     """Kolmogorov-Arnold Network model improved by using a self-organizing map
105     that is filled with data distribution information.
106
107     This model is supposed to be trained in a continual learning settings it
108     requires initialization of the SOM layer with samples from the base dataset
109     before training begins.
110     """
111
112     def __init__(
113         self,
114         input_dim: int = 2048,
115         hidden_dim: int = 64,
116         output_dim: int = 200,
117         som_variance_init: float = 0.5,
118         som_variance_lr: float = 0.05,
119     ):
120         super().__init__()
121         self.som = SOMLayer(input_dim, hidden_dim, som_variance_init, som_variance_lr)
122         self.classifier = KANLayer(hidden_dim, output_dim)
123         # self.classifier = torch.nn.Linear(hidden_dim, output_dim)
124
125     def forward(self, x: torch.Tensor) -> torch.Tensor:
126         similarities = self.som(x)
127         preds = self.classifier(similarities)
128         return preds
129
130     def som_init(
131         self,
132         trainloader: torch.utils.data.DataLoader,
133         device: torch.device = torch.device("cpu"),
134     ) -> None:
135         """Perform initial optimization of the SOM layer by moving prototypes
136         towards samples from dataset's base classes"""
137         for batch_inputs, _ in tqdm(
138             trainloader, desc="SOMLayer initialization", leave=False
139         ):
140             self.som.update(batch_inputs.to(device))

```